

Tercera serie

FRANCISCO BARRIOS

0900 21 1146

CENSO

// config/db.js

```
const mongoose = require('mongoose');
```

```
// Requiere dotenv para que Node.js lea el MONGO_URI de tu archivo .env
```

```
require('dotenv').config();
```

```
const connectDB = async () => {
```

```
  try {
```

```
    // Intenta conectar usando la variable MONGO_URI
```

```
    await mongoose.connect(process.env.MONGO_URI);
```

```
    // Si tiene éxito, imprime el mensaje clave
```

```
    console.log('✅ MongoDB conectado exitosamente.');
```

```
  } catch (err) {
```

```
    // Si falla, imprime el error y cierra la aplicación
```

```
    console.error('❌ Error de conexión a MongoDB:', err.message);
```

```
    process.exit(1);
```

```
  }
```

```
};
```

```
module.exports = connectDB;
```

```
// models/Censo.js

const mongoose = require('mongoose');

// Define el esquema (Schema) del Censo
const CensoSchema = new mongoose.Schema({

  // Nota: El campo ID es el "_id" que Mongoose y MongoDB crean automáticamente.

  nombre: {
    type: String,
    required: [true, 'El nombre es obligatorio.'],
    trim: true
  },
  edad: {
    type: Number,
    required: [true, 'La edad es obligatoria.'],
    min: [0, 'La edad no puede ser negativa.']
  },
  sexo: {
    type: String,
    required: [true, 'El sexo es obligatorio.'],
    enum: {
      values: ['Masculino', 'Femenino', 'Otro'],
      message: '{VALUE} no es un sexo válido.'
    }
  },
},
```

```
    localidad: {  
      type: String,  
      required: [true, 'La localidad es obligatoria.'],  
      trim: true  
    }  
  }, {  
    timestamps: true // Agrega campos createdAt y updatedAt  
  });
```

```
// Exporta el modelo (la colección se llamará "censos" automáticamente)  
module.exports = mongoose.model('Censo', CensoSchema);
```

```
// routes/censoRoutes.js  
  
const express = require('express');  
const router = express.Router();  
const Censo = require('../models/Censo');  
  
// *** 4. POST /censo: Agregar un nuevo registro. ***  
router.post('/', async (req, res) => {  
  try {  
    const nuevoRegistro = new Censo(req.body);
```

```

const registroGuardado = await nuevoRegistro.save();

// 201 Created - Devuelve el objeto recién creado
res.status(201).json(registroGuardado);
} catch (error) {

// 400 Bad Request - Errores de validación (ej. falta un campo requerido)
res.status(400).json({
  msg: 'Error al agregar registro. Verifique los campos: nombre, edad, sexo,
localidad.',
  error: error.message
});
}
});

// *** 5. GET /censo: Listar todos los registros del censo. ***
router.get('/', async (req, res) => {
  try {
    const registros = await Censo.find();

    // 200 OK - Devuelve el array de todos los registros
    res.json(registros);
  } catch (error) {
    // 500 Internal Server Error
    res.status(500).json({ msg: 'Error al obtener todos los registros.', error:
error.message });
  }
}

```

```
});
```

```
// *** 6. GET /censo/<id>: obtener registro específico. ***
```

```
router.get('/:id', async (req, res) => {
```

```
  try {
```

```
    // Busca un documento por su ID (_id)
```

```
    const registro = await Censo.findById(req.params.id);
```

```
    // 404 Not Found - Si el ID es válido pero no existe el registro
```

```
    if (!registro) {
```

```
      return res.status(404).json({ msg: `Registro con ID ${req.params.id} no  
      encontrado.` });
```

```
    }
```

```
    // 200 OK - Devuelve el objeto individual
```

```
    res.json(registro);
```

```
  } catch (error) {
```

```
    // Maneja errores de formato de ID (ej. si el ID tiene un formato inválido)
```

```
    res.status(500).json({ msg: 'Error al obtener registro específico. El ID podría ser  
    inválido.', error: error.message });
```

```
  }
```

```
});
```

```
module.exports = router;
```

```
{
  "name": "censowebbservice",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "dotenv": "^17.2.2",
    "express": "^5.1.0",
    "mongoose": "^8.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.10"
  }
}
```

```
// server.js

const express = require('express');

const connectDB = require('./config/db'); // 1. Importa la función de conexión
require('dotenv').config();

// Importa las rutas del censo (la lógica POST/GET)
const censoRoutes = require('./routes/censoRoutes');

// 2. Ejecuta la conexión a la base de datos
connectDB();

const app = express();

// Middleware esencial: permite a Express leer JSON en el cuerpo de las peticiones
// (necesario para el POST)
app.use(express.json());

// Monta las rutas: todas las peticiones a /censo serán manejadas por censoRoutes
app.use('/censo', censoRoutes);

// Endpoint simple de prueba
app.get('/', (req, res) => {
  res.send('Servicio Web de Censo activo. Prueba enviando peticiones a /censo.');
```

```
});

// Obtiene el puerto del .env (3000)
```

```
const PORT = process.env.PORT || 3000;
```

// 3. Inicia el servidor Express

```
app.listen(PORT, () => {
```

```
  console.log(` 🚀 Servidor Express corriendo en http://localhost:${PORT} `);
```

```
});
```

MONGO\_URI=mongodb://localhost:27017/censowebsservice

PORT=3000

config > JS db.js > ...

```
1  // config/db.js
2  const mongoose = require('mongoose');
3  // Requiere dotenv para que Node.js lea el MONGO_URI de tu archivo .env
4  require('dotenv').config();
5
6  const connectDB = async () => {
7    try {
8      // Intenta conectar usando la variable MONGO_URI
9      await mongoose.connect(process.env.MONGO_URI);
10
11     // Si tiene éxito, imprime el mensaje clave
12     console.log('✅ MongoDB conectado exitosamente.');
```



```
JS server.js  {} package.json  JS censoRoutes.js  {} package-lock.json  JS Censo.js  X  .env
models > JS Censo.js > CensoSchema > sexo
1  // models/Censo.js
2  const mongoose = require('mongoose');
3
4  // Define el esquema (Schema) del Censo
5  const CensoSchema = new mongoose.Schema({
6    // Nota: El campo ID es el "_id" que Mongoose y MongoDB crean automáticamente.
7
8    nombre: {
9      type: String,
10     required: [true, 'El nombre es obligatorio.'],
11     trim: true
12   },
13   edad: {
14     type: Number,
15     required: [true, 'La edad es obligatoria.'],
16     min: [0, 'La edad no puede ser negativa.'],
17   },
18   sexo: {
19     type: String,
20     required: [true, 'El sexo es obligatorio.'],
21     enum: {
22       values: ['Masculino', 'Femenino', 'Otro'],
23       message: '{VALUE} no es un sexo válido.'
24     }
25   },
26   localidad: {
27     type: String,
```

JS server.js X {} package.json JS censoRoutes.js X {} package-lock.json JS Censo.js .env JS db.js

C:\Users\pac\_p\OneDrive\Documentos\CensoWebService\server.js

```
1 // routes/censoRoutes.js
2 const express = require('express');
3 const router = express.Router();
4 const Censo = require('../models/Censo');
5
6 // *** 4. POST /censo: Agregar un nuevo registro. ***
7 router.post('/', async (req, res) => {
8   try {
9     const nuevoRegistro = new Censo(req.body);
10
11     const registroGuardado = await nuevoRegistro.save();
12
13     // 201 Created - Devuelve el objeto recién creado
14     res.status(201).json(registroGuardado);
15   } catch (error) {
16     // 400 Bad Request - Errores de validación (ej. falta un campo requerido)
17     res.status(400).json({
18       msg: 'Error al agregar registro. Verifique los campos: nombre, edad, sexo, localidad.',
19       error: error.message
20     });
21   }
22 });
23
24
25 // *** 5. GET /censo: Listar todos los registros del censo. ***
26 router.get('/', async (req, res) => {
27   try {
28     const registros = await Censo.find();
29
30     // 200 OK - Devuelve el array de todos los registros
31     res.json(registros);
32   } catch (error) {
33     // 500 Internal Server Error
```

EXPLORER ... JS server.js {} package.json X JS censoRoutes.js JS Censo.js .env JS db.js

▼ CENSOWEBSERVICE

- ▼ config
- JS db.js
- ▼ models
- JS Censo.js
- > node\_modules
- ▼ routes
- JS censoRoutes.js
- .env
- { } package-lock.json
- { } package.json
- JS server.js

{ } package.json > {} scripts > test

```
1 {
2   "name": "censoweb-service",
3   "version": "1.0.0",
4   "description": "",
5   "main": "server.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1",
8     "dev": "nodemon server.js"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "dependencies": {
14    "dotenv": "^17.2.2",
15    "express": "^5.1.0",
16    "mongoose": "^8.18.2"
17  },
18}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\pac\_p\OneDrive\Documentos\CensoWebService> npm run dev

log

PS C:\Users\pac\_p\OneDrive\Documentos\CensoWebService> npm run dev

> censoweb-service@1.0.0 dev

> nodemon server.js

[nodemon] 3.1.10

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): \*.\*

[nodemon] watching extensions: js,mjs,cjs,json

[nodemon] starting `node server.js`

[dotenv@17.2.2] injecting env (2) from .env -- tip: ⚠ override existing env vars with { override: true }

[dotenv@17.2.2] injecting env (0) from .env -- tip: ⚠ auto-backup env with Radar: <https://dotenvx.com/radar>

✖ Servidor Express corriendo en http://localhost:3000

✖ Error de conexión a MongoDB: connect ECONNREFUSED ::1:27017, connect ECONNREFUSED 127.0.0.1:27017

[nodemon] app crashed - waiting for file changes before starting...

> OUTLINE

> TIMELINE

JS server.js {} package.json X JS censoRoutes.js {} package-lock.json JS Censo.js .env JS

{ } package.json > {} dependencies

```
1 {
2   "name": "censoweb-service",
3   "version": "1.0.0",
4   "description": "",
5   "main": "server.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1",
8     "dev": "nodemon server.js"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "dependencies": {
14    "dotenv": "^17.2.2",
15    "express": "^5.1.0",
16    "mongoose": "^8.18.2"
17  },
18  "devDependencies": {
19    "nodemon": "^3.1.10"
20  }
21}
```

JS server.js > ...

```
1 // server.js
2 const express = require('express');
3 const connectDB = require('./config/db'); // 1. Importa la función de conexión
4 require('dotenv').config();
5
6 // Importa las rutas del censo (la lógica POST/GET)
7 const censoRoutes = require('./routes/censoRoutes');
8
9 // 2. Ejecuta la conexión a la base de datos
10 connectDB();
11
12 const app = express();
13
14 // Middleware esencial: permite a Express leer JSON en el cuerpo de las peticiones (necesario para el POST)
15 app.use(express.json());
16
17 // Monta las rutas: todas las peticiones a /censo serán manejadas por censoRoutes
18 app.use('/censo', censoRoutes);
19
20 // Endpoint simple de prueba
21 app.get('/', (req, res) => {
22 |   res.send('Servicio Web de Censo activo. Prueba enviando peticiones a /censo.');
```

---

```
23 });
24
25 // Obtiene el puerto del .env (3000)
26 const PORT = process.env.PORT || 3000;
27
28 // 3. Inicia el servidor Express
29 app.listen(PORT, () => {
30 |   console.log(`🚀 Servidor Express corriendo en http://localhost:${PORT}`);
31 | });
```